

Approches classiques de l'IA

Chapitre 2 : Agents intelligents

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences

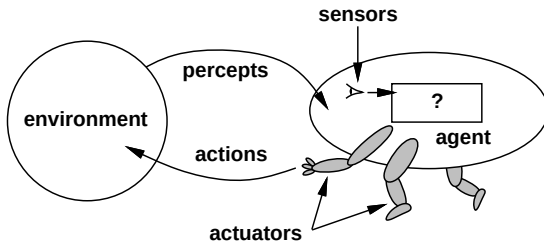


Chapitre 2. Agents intelligents

- 1 Agents et environnements
- 2 La notion de rationalité
- 3 La nature des environnements
- 4 Programmes d'agents
- 5 Résumé du chapitre

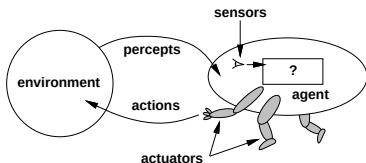
Agents et environnements

Agents et environnements



Intuitivement, un **agent** est quelque chose qui perçoit son **environnement** grâce à des **senseurs** et agit sur cet environnement grâce à des **actuateurs**.

Agents et environnements



Exemples :

- Un agent robot peut avoir des caméras, des lasers, etc. comme senseurs et différents moteurs comme actuateurs.
- Un agent logiciel reçoit des entrées clavier, des paquets réseau, le contenu d'un fichier, etc. (entrées $\simeq$ senseurs) et agit via l'écran, l'écriture dans un fichier ou l'envoi de paquets réseau (sorties $\simeq$ actuateurs).

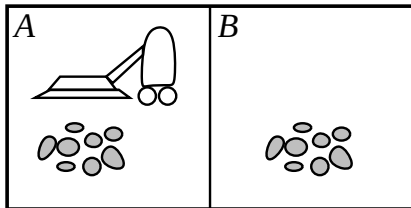
Séquence des perceptions

Une **perception** sont les entrées que l'agent perçoit à un moment donné.

La **séquence des perceptions** est l'historique complet de tout ce que l'agent a déjà perçu.

Le comportement d'un agent est décrit par une **fonction d'agent** (ou **programme d'agent**) qui associe une action à chaque séquence de perceptions.

Exemple : le monde du robot aspirateur



Environnement très simple : composé de 2 endroits (les carrés *A* et *B*). Un carré peut être soit propre, soit sale.

Perceptions : localisation et contenu (exemple [*A, Propre*]).

Actions : *Gauche*, *Droite*, *Aspire*, *Rien*.

De multiples agents pourraient être définis, mais **quelle est une bonne fonction d'agent** ? Comment définir un comportement **rationnel** ?

La notion de rationalité

La notion de rationalité

Un **agent rationnel** est un agent qui réalise l'action adéquate, mais comment déterminer qu'une action est meilleure qu'une autre ?

Nous allons répondre à cette question en considérant les **conséquences sur son environnement** du comportement d'un agent, via une **mesure de performance**.

Mesure de performance

Exemple : deux mesures de performance possibles pour le robot aspirateur

- (a) quantité de poussière retirée par tranche de 8 heures ;
- (b) donner 1 point pour chaque carré nettoyé par unité de temps et (éventuellement) une pénalité pour l'électricité consommée ou le bruit généré.

Que pensez-vous de ces 2 mesures ? (Avec un agent rationnel, “ce qu'on demande est ce qu'on obtient”).

Mesure de performance

Si on utilise la mesure (a) (quantité de poussière retirée par tranche de 8h), une fonction d'agent qui la maximise pourrait être de nettoyer la saleté, de la rejeter sur le sol, puis de renettoyer, et ainsi de suite !

Il est en général plus efficace de concevoir des mesures de performances en fonction de ce que l'on veut obtenir en réalité dans l'environnement.

La mesure (b) donne une meilleure évaluation de la performance car elle exprime le niveau de propreté moyen en fonction du temps.

Rationalité

Définition

Pour n'importe quelle séquence de perceptions possible, un **agent rationnel** sélectionne une action qui est censée maximiser sa mesure de performance, en fonction des informations fournies par la séquence de perceptions et sa connaissance initiale de l'environnement.

La rationalité dépend donc de quatre choses :

- La mesure de performance ;
- La connaissance initiale que possède l'agent par rapport à l'environnement ;
- Les actions que l'agent peut réaliser ;
- La séquence des perceptions de l'agent à l'instant t .

Rationalité

Exemple : soit la fonction d'agent REFLEX-VACUUM-AGENT pour le robot aspirateur. Définit-elle un agent rationnel ?

```
function REFLEX-VACUUM-AGENT([location, status]) returns an action  
  
if status = Sale then return Aspire  
else if location = A then return Droite  
else if location = B then return Gauche
```

Rationalité

Exemple : soit la fonction d'agent REFLEX-VACUUM-AGENT pour le robot aspirateur. Définit-elle un agent rationnel ?

```
function REFLEX-VACUUM-AGENT([location, status]) returns an action  
  
if status = Sale then return Aspire  
else if location = A then return Droite  
else if location = B then return Gauche
```

Cela dépend ! Il faut définir la mesure de performance, ce qui est connu à propos de l'environnement et quels sont les senseurs et les actuateurs de l'agent.

Rationalité

Soit les hypothèses suivantes :

- La mesure de performance donne 1 point pour chaque carré nettoyé lors d'une unité de temps, sur une période de 1000 unités de temps.
- La "géographie" de l'environnement (carré *A* à gauche du carré *B*) est connue *a priori* mais la distribution de la saleté et la localisation initiale de l'environnement ne l'est pas.
- Les seules actions possibles sont *Gauche*, *Droite* et *Aspire*.
- Un carré propre reste propre et l'action *Aspire* nettoie le carré courant.
- Les actions *Droite* et *Gauche* déplacent l'agent à droite ou à gauche sauf si cela mènerait l'agent hors de l'environnement (dans ce cas, l'agent reste où il est).
- L'agent perçoit correctement sa localisation et si celle-ci contient de la saleté.

Rationalité

Sous ces hypothèses, pensez-vous que l'agent est rationnel, c'est à dire que sa performance espérée est au moins aussi bonne que celle de n'importe quel autre agent ?

Rationalité

Sous ces hypothèses, pensez-vous que l'agent est rationnel, c'est à dire que sa performance espérée est au moins aussi bonne que celle de n'importe quel autre agent ?

Oui ! car le score maximal pour cette mesure correspond au nombre de carrés sales au départ !

Pourtant, quand les cases sales seront nettoyées, cet agent va osciller de droite à gauche indéfiniment.

Un meilleur agent s'arrêterait dès qu'il est sûr que tout est propre, ce qui pourrait être atteint en incluant dans la mesure une pénalité pour chaque mouvement.

Rationalité = Omniscience ?

- Un agent **omniscient** connaît le score réel de ses actions et peut agir en conséquence.
- L'omniscience n'est pas (toujours) possible en fonction de l'environnement.
- Les perceptions peuvent **ne pas fournir toutes** les informations utiles.

Exemple : Je me promène en rue et vois un ami de l'autre côté. Il n'y a pas de trafic et j'ai du temps devant moi. Je décide de traverser la rue. Au même moment, à 33000 pieds, un avion perd un morceau de carlingue et je suis écrasé avant d'avoir atteint l'autre côté de la rue. Était-ce irrationnel de traverser ?

Rationalité $\neq$ Omniscience

Rationalité

Notre définition de rationalité n'implique donc pas l'omniscience car le choix rationnel ne dépend que de l'information connue (séquence des perceptions et information initiale).

Cependant, nous devons nous assurer que l'agent ne s'engage pas dans des activités stupides, par manque de connaissance.

Exemple : si l'agent ne regarde pas des deux côtés avant de traverser l'avenue Maistriau, sa séquence de perceptions ne lui sera pas utile pour déterminer si un camion arrive à grande vitesse.

Récolte d'informations

Notre définition de la rationalité permet-elle de telles actions ?

Non car,

- il n'est pas rationnel de traverser la rue avec une séquence de perceptions sans information utile ;
- un agent rationnel choisira l'action *Regarder* avant de traverser, parce que cette action aide à maximiser la performance espérée.

Réaliser une action dans le **but de modifier les perceptions futures** est appelé la **récolte d'informations** (information gathering).

Exemple : l'**exploration** du robot aspirateur dans un environnement inconnu.

Apprentissage

Notre définition implique également d'un agent rationnel qu'il **apprenne** le plus possible de ce qu'il perçoit, pour compenser des connaissances partielles ou erronées.

- configuration initiale de l'agent $\simeq$ connaissance initiale ;
- connaissance peut être modifiée / augmentée ;
- capacité d'apprentissage permet de concevoir un seul agent performant dans des environnements variés.

Exemple : un robot aspirateur qui apprend à prévoir où et quand de la saleté va apparaître.

La nature des environnements

PEAS

Pour rappel, pour concevoir un agent rationnel, il faut connaître 4 éléments (PEAS) :

- mesure de **P**erformance
- **E**nvironnement
- **A**ctionneurs
- **S**enseurs

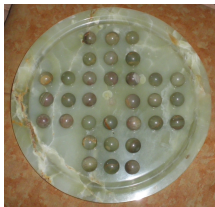
Le **type de l'environnement** va nous permettre de savoir quelle méthode pourra être utilisée. Ce type est défini par une série de critères.

Types d'environnements : observabilité

Observabilité :

- **complètement observable** : les senseurs détectent *tous* les aspects qui sont *utiles* au choix d'une action.
- **non observable** : l'agent n'a pas de senseurs.
- **partiellement observable** : dû au bruit ou à l'inexactitude des perceptions ; ou parce certains états de l'environnement sont inaccessibles via les senseurs (*e.g.*, le robot aspirateur qui ne perçoit la poussière que localement, le taxi autonome qui ne sait pas ce que les autres conducteurs pensent).

Types d'environnements : observabilité



	Solitaire	Backgammon	Taxi
Observable ??			



Types d'environnements : observabilité

	Solitaire	Backgammon	Taxi
Observable ??	Oui	Oui	Part.



Remarque : au backgammon, au moment de prendre sa décision, les dés sont déjà jetés et l'agent a donc toutes les informations nécessaires pour prendre sa décision.

Types d'environnements : déterminisme

Déterministe vs Stochastique :

- **Déterministe** : le hasard n'intervient pas dans l'environnement.
- **Stochastique** : le hasard peut intervenir (i.e., l'état suivant de l'environnement n'est pas uniquement déterminé par l'état courant et l'action exécutée par l'agent).

Types d'environnements : déterminisme

	Solitaire	Backgammon	Taxi
Observable	Oui	Oui	Part.
Déterministe ??			

Types d'environnements : déterminisme

	Solitaire	Backgammon	Taxi
Observable	Oui	Oui	Part.
Déterministe ??	Oui	Non	Non

Types d'environnements : statique ou dynamique

Statique vs Dynamique :

- si l'environnement peut changer pendant que l'agent prend sa décision, alors il est **dynamique**.
- autrement il est **statique** (ne change pas avec le temps).

Types d'environnements : statique ou dynamique

	Solitaire	Backgammon	Taxi
Observable	Oui	Oui	Part.
Déterministe	Oui	Non	Non
Statique ??			

Types d'environnements : statique ou dynamique

	Solitaire	Backgammon	Taxi
Observable	Oui	Oui	Part.
Déterministe	Oui	Non	Non
Statique ??	Oui	Oui	Non

Types d'environnements : discret ou continu

Discret vs **Continu**. Cette distinction s'applique

- à l'**état** de l'environnement,
- à la façon dont le **temps** est manipulé,
- aux **perceptions** et **actions** de l'agent.

Exemple : l'environnement du jeu d'échecs a un nombre fini d'états distincts, ainsi qu'un ensemble discret de perceptions et d'actions. Le taxi autonome évolue dans un environnement à états continus (vitesse, localisation du taxi et des autres véhicules) et les actions sont continues (angles directions, niveau d'accélération, etc.).

Types d'environnements : discret ou continu

	Solitaire	Backgammon	Taxi
Observable	Oui	Oui	Part.
Déterministe	Oui	Non	Non
Statique	Oui	Oui	Non
Discret ??			

Types d'environnements : discret ou continu

	Solitaire	Backgammon	Taxi
Observable	Oui	Oui	Part.
Déterministe	Oui	Non	Non
Statique	Oui	Oui	Non
Discret ??	Oui	Oui	Non

Types d'environnements : solo ou multi

Agent unique vs Multi-agents.

Exemple : un agent qui résout un mot croisé par lui-même est dans un environnement “agent unique”, tandis qu’un agent qui joue aux échecs est dans un environnement “2-agents”.

On distingue les environnements multi-agents **compétitifs** (échecs) des multi-agents **coopératifs** (e.g., taxi autonome : le but de chacun est d’éviter les collisions).

Types d'environnements

	Solitaire	Backgammon	Taxi
Observable	Oui	Oui	Part.
Déterministe	Oui	Non	Non
Statique	Oui	Oui	Non
Discret	Oui	Oui	Non
Multi-agents ??			

Types d'environnements

	Solitaire	Backgammon	Taxi
Observable	Oui	Oui	Part.
Déterministe	Oui	Non	Non
Statique	Oui	Oui	Non
Discret	Oui	Oui	Non
Multi-agents ??	Non	Oui	Oui

Types d'environnements

Remarques :

- Le type d'environnement influence la conception d'agents.
- Le cas le plus difficile : partiellement observable, stochastique, dynamique, continu et multi-agents.
- La dépêche concernant le projet de voitures autonomes de Google du 9 octobre 2010 a donc impressionné à l'époque¹.

1. <http://googleblog.blogspot.com/2010/10/what-were-driving-at.html>

Programmes d'agents

Programmes d'agents

Travail principal de l'IA : concevoir un **programme d'agent** qui associe une action aux perceptions.

Les programmes d'agents seront décrits en **pseudo-code** sous la forme d'une fonction qui prend la perception courante en entrée et qui retourne une action.

Remarque concernant le pseudo-code : le mot clef **persistant** est utilisé pour une variable qui reçoit une valeur initiale au premier appel de la fonction, et qui reste accessible lors des appels suivants ($\simeq$ variable globale).

Programmes d'agents

Exemple de types de programmes d'agents (par ordre de généralité croissante) :

- agents-réflexe simples ;
- agents-réflexe avec modèle ;
- agents basés sur des objectifs ;
- agents basés sur l'utilité.

Agents-réflexe simples

(Simple reflex agents)

Principe : choisit une action sur la base de la perception *courante* uniquement (ignore l'historique) grâce à des règles “*condition-action*”.

Exemple : le robot aspirateur vu précédemment

```
function REFLEX-VACUUM-AGENT([location, status] ) returns action  
  
if status = Sale then return Aspire  
else if location = A then return Droite  
else if location = B then return Gauche
```

Agents-réflexe simples

(Simple reflex agents)

De manière plus générale :

```
function SIMPLE-REFLEX-AGENT(percept) returns action  
persistent: rules, ensemble de règles condition-action  
  
state  $\leftarrow$  INTERPRET-INPUT(percept)  
rule  $\leftarrow$  RULE-MATCH(state, rules)  
action  $\leftarrow$  rule.ACTION  
returns action
```

Agents-réflexe simples

(Simple reflex agents)

Les agents-réflexe simples ne fonctionnent que si la décision correcte peut être prise uniquement sur base de la perception courante. C'est le cas des environnements complètement observables.

Dans des environnements partiellement observables, les boucles infinies sont souvent inévitables pour des agents-réflexe simples.

Exemple : si un robot aspirateur détecte s'il y a de saleté mais **pas** sa propre localisation. Il y a donc 2 perceptions possibles : [*Sale*] et [*Propre*]. Il peut retourner "*Aspire*" en cas de saleté, mais que doit-il faire en réponse à [*Propre*] ? Bouger à gauche va échouer (indéfiniment) s'il se trouve en A.

Solution possible : randomisation, mais performance ↘ .

Agents-réflexe avec modèle

(Model-based reflex agents)

Principe : gérer l'observabilité partielle en modélisant l'environnement :

- Maintenir un **modèle interne** de l'environnement qui dépend de l'historique et reflète une partie des aspects de l'environnement que l'on ne peut pas (ou plus) voir.
Exemple (taxi) : pour changer de bande de circulation, se souvenir des autres voitures vues, même si invisibles pour le moment.
- Cela permet de tenir compte du “passé” (i.e., historique complet des perceptions).

Agents-réflexe avec modèle

(Model-based reflex agents)

function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** action

persistent: *model*, modèle de l'environnement

state, état courant de l'environnement, selon l'agent

action, l'action la plus récente (initialement vide)

rules, ensemble de règles condition-action

state $\leftarrow$ UPDATE-STATE(*state*, *action*, *percept*, *model*)

rule $\leftarrow$ RULE-MATCH(*state*, *rules*)

action $\leftarrow$ *rule*.ACTION

returns *action*

Agents basés sur des objectifs

(Goal-based agents)

Principe : se doter d'objectifs permet de décider en tenant compte du "futur"

- Atteindre un objectif peut requérir une (longue) séquence d'actions.
- Permet des décisions très différentes que celles obtenues via des règles "condition-action" (agents-réflexe), car prend le futur en considération
- peut être combiné avec un modèle.

Exemples :

- objectifs taxi autonome = destination du passager, sécurité ;
- objectif jeu d'échec = gagner la partie.

C'est le type d'agents que nous utiliserons au chapitre 3.

Agents basés sur l'utilité

(Utility-based agents)

Principe : mesurer un niveau de satisfaction.

- Uniquement objectifs : pas toujours suffisant pour générer des comportements de qualité dans beaucoup d'environnements. Un objectif offre une vision binaire de la satisfaction.
- **Fonction d'utilité** : mesure le niveau de satisfaction de l'agent (modèle interne de la mesure de performance).

C'est le type d'agents que nous utiliserons aux chapitres 4 et 5.

Agents qui apprennent

(Learning agents)

L'**apprentissage** est la capacité pour un agent d'améliorer ses performances.

Chacun des 4 modèles de bases vus ci-avant peuvent être dotés de capacités d'apprentissage (e.g., modification des règles pour un agent réflexe).

L'AA "Introduction au Deep Learning" vous initiera à l'apprentissage automatique.

Résumé du chapitre

Résumé du chapitre

Ce chapitre présente schématiquement les types d'agents les plus fréquents et leurs environnements.

- un **agent** perçoit et agit dans son environnement. La **fonction d'agent** spécifie l'action effectuée par l'agent en réponse à une séquence de perceptions.
- la **mesure de performance** évalue le comportement d'un agent dans son environnement. Un **agent rationnel** agit pour maximiser la valeur espérée de la mesure de performance, étant donné la séquence de perceptions courante.
- La spécification d'un **environnement de travail** inclut la mesure de performance, l'environnement externe, les acteurs et les senseurs.
- Les environnements de travail varient selon différentes dimensions (observabilité, nombre d'agents, etc.)

Résumé du chapitre

- Le **programme d'agent** implémente la fonction d'agent. Il existe quelques designs de base pour les programmes d'agent. Ces designs varient en efficacité, flexibilité etc. Le design approprié dépend de la nature de l'environnement.
- Les **agents-réflexe simples** répondent directement à la perception courante ; les **agents-réflexe avec modèle** maintiennent un état interne pour refléter les parties du monde qui ne sont pas observables ; les **agents basés sur des objectifs** agissent pour atteindre leurs objectifs ; les **agents basés sur l'utilité** essaient de maximiser leur niveau de satisfaction (fonction d'utilité) espéré.
- Tous les types d'agents peuvent améliorer leurs performances grâce à l'**apprentissage**.